프로젝트 보고서

MoGe2 기반 **Depth** 정보를 활용한 **FAST-LIVO2** 유리창 표현 보완 연구

(LiDAR-Camera 기반 유리창 검출 및 후처리 보정 연구)

과목명**:** 창의설계프로젝트1

프로젝트명**:** MoGe-2 기반 Depth 정보를 활용한 FAST-LIVO2 유리창 인식 보완

학과**:** 전자공학부

학번**:** 20210327, 20210338, 20210747

이름**:** 김찬효, 김태준, 윤동식

지도교수**:** 전명환

제출일**:** 2026년 06월 23일

목차

1. 제목 및 초록

1.1 초록

2. 서론

2.1 프로젝트의 배경 및 필요성

2.2 해결하고자 하는 문제

2.3 프로젝트의 목표 및 기대 효과

3. 관련 연구 또는 기존 사례 조사

3.1 LiDAR-Camera 기반 SLAM 기술

3.2 이미지 기반 Depth Estimation 기술

3.3 투명 물체 인식의 기존 한계점

3.4 본 프로젝트와의 차별점

4.제안 방법

4.1 하드웨어 구성,회로 및 기구 설계

4.2 주요 하드웨어 선정

4.3 기구 설계

4.4 하드웨어 회로 설계

4.5 소프트웨어 아키텍쳐 구성

4.6 전체 시스템 흐름도

5. 구현 내용

5.1 현재까지 구현한 내용

5.2 제작 과정 및 주요 설계 변경 사항

6. 실험 및 결과

6.1 성능 검증 방법

6.2 실험 조건 및 평가 기준

6.3 현재까지의 실험 결과

6.4 기타 실패 사례, 문제점 분석

7. 결론 및 향후 계획

7.1 1학기 동안의 진행 내용 요약

7.2 현재 한계점

7.3 2학기 또는 향후 개선 계획

8. 참고문헌

1. 제목 및 초록

1.1 초록

본 프로젝트는 LiDAR-Camera 기반 SLAM 시스템인 FAST-LIVO2에서 유리창과 같은 투명 물체가 RGB 기반 시각화 결과에 충분히 반영되지 않는 문제를 보완하기 위해 진행되었다. 유리창은 RGB 이미지에서는 명확히 확인되지만, LiDAR 점군에서는 레이저 신호의 투과 및 불안정한 반사로 인해 실제 유리면이 형성되지 않는 치명적인 한계가 존재한다.

이를 극복하기 위해 본 연구에서는 ROS Bag 파일에서 추출한 RGB 이미지에 MoGe-2 모델을 적용하여 프레임별 절대 뎁스를 추출하였다. 나아가 생성된 뎁스 정보를 FAST-LIVO2의 C++ 맵핑 로직 내부로 통합하고, LiDAR 평면 뼈대를 기준으로 뎁스 노이즈를 깎아내는 최적화 파이프라인을 성공적으로 구축하였다.

결과적으로 기존 FAST-LIVO2 단일 구동 시보다 유리창 영역을 조밀하게 복원할 수 있음을 확인하였다. 단, 투명 매질의 본질적인 투과 현상과 곡률 피팅의 한계가 관찰되었으며, 이를 완벽히 해결하기 위해 향후 Grounded-SAM 2 기반의 2D 시맨틱 세그멘테이션 마스크를 3D 공간으로 투영하는 2차 융합 구조로 개선할 계획이다.

2. 서론

2.1 프로젝트의 배경 및 필요성

SLAM은 자율주행 시스템이 주변 환경을 인식하고 위치를 추정하는 핵심 기술이다. 특히 LiDAR, IMU, Camera를 융합하는 FAST-LIVO2는 다양한 환경에서 정밀한 매핑을 수행한다. 그러나 실내 환경에 다수 존재하는 유리창, 거울 등의 투명 및 반사성 물체는 센서 신호가 투과되거나 난반사되어 SLAM 결과에 안정적으로 반영되지 못한다. 따라서 맵의 신뢰성을 높이기 위해 기존 결과를 보완할 수 있는 추가적인 시각적 뎁스 융합 기술이 필수적이다.

2.2 해결하고자 하는 문제

본 프로젝트의 핵심 과제는 FAST-LIVO2 구동 시 유리창 영역이 뚫린 공간으로 인식되는 기하학적 붕괴 현상을 해결하는 것이다. 아래 그림과 같이 육안으로는 유리가 막혀있음을 알 수 있으나, LiDAR 점군에서는 유리면이 형성되지 않아 맵이 누락된다.



이를 해결하기 위해 RGB 이미지에 MoGe-2를 적용하여 절대 뎁스 정보를 생성하고, 이를 LiDAR 뼈대와 결합하여 부족한 유리창 영역을 메우는 방식을 제안한다.

2.3 프로젝트의 목표 및 기대 효과

본 프로젝트는 MoGe-2의 절대 뎁스 정보와 FAST-LIVO2의 궤적 및 맵 데이터를 융합하여 투명 물체의 3D 표현 한계를 보완하는 것을 목표로 한다.

1. 기존 FAST-LIVO2에서 누락되던 투명 객체 영역을 고밀도 점군으로 복원한다.
2. ROS Bag 이미지와 AI 뎁스 추정 모델을 결합하는 최적화 융합 파이프라인을 구축한다.
3. 구축된 융합 맵을 바탕으로 무결점의 3D 가우시안 스플래팅 기반 환경 렌더링을 위한 토대를 마련한다.

3. 관련 연구 및 기존 사례 조사

3.1 LiDAR-Camera 기반 **SLAM** 기술

최근의 SLAM은 LiDAR, IMU, Camera를 모두 활용하는 센서 퓨전 방식으로 발전하였다. FAST-LIVO2는 이종 센서를 강결합하여 높은 성능을 내지만, 유리와 같은 투명 매질에서는 레이저가 투과되어 점군이 형성되지 않는 물리적 한계가 뚜렷하다.

3.2 이미지 기반 **Depth Estimation** 기술

단일 RGB 이미지로부터 픽셀의 깊이를 추정하는 기술이다. 본 연구에서 활용한 MoGe-2는 단순한 상대적 깊이를 넘어, 장면의 3차원 기하 정보를 분석하여 미터(m) 단위의 절대 뎁스를 출력하는 파운데이션 모델이다. 이를 통해 LiDAR가 놓친 유리창의 뎁스를 시각적으로 유추할 수 있다.

3.3 투명 물체 인식의 기존 한계점

- LiDAR의 한계: 빛의 투과 특성으로 인해 유리 표면의 점군 생성 불가.
- Camera SLAM의 한계: 텍스처(무늬)가 없는 투명한 유리 표면에서 특징점 추출 및 시차 매칭 실패.
- 단안 뎁스 모델의 한계: 뎁스를 추정하더라도 투명한 특성 탓에 유리 표면이 아닌 창문 너머의 배경 거리를 출력해버리는 투과 현상 발생.

3.4 본 프로젝트와의 차별점

기존 방식은 FAST-LIVO2를 단독 실행하는 데 그치지만, 본 프로젝트는 AI 비전 모델이 추출한 뎁스 데이터를 C++ 기반의 LiDAR SLAM 맵핑 로직 내부로 끌고 들어와, 평면 뼈대를 기준으로 노이즈를 깎아내는 정교한 최적화를 수행한다는 점에서 독보적인 차별성을 가진다.

4. 제안 방법

4.1 하드웨어 구성 및 주요 장비 선정

이종 센서 간의 정밀한 퓨전을 위해 엣지 컴퓨팅 기반의 하드웨어를 아래와 같이 구성하였다.



- LiDAR (Livox Horizon): 비반복적 스캐닝으로 고밀도 점군 제공, FAST-LIVO와 최적의 호환성.
- Camera (ZED X Stereo): MoGe-2의 정밀한 단안 3D PCD 추출을 위한 고해상도 이미지 취득.
- Edge PC (Jetson AGX Orin 64GB): 딥러닝 추론 및 ROS 2 병렬 처리를 위한 고성능 엣지 컴퓨터.
- Battery (파워크래프트 DMBP-LF128): 순간적인 피크 전류에 대비한 안정적인 전원 공급 및 시스템 다운 방지.

4.2 기구 및 회로 설계

- 시차 최소화 배치: LiDAR 점군을 이미지에 투영할 때 발생하는 기하학적 오차를 줄이기 위해, 카메라를 라이다 직상단에 최대한 밀착시켜 수직 축을 일치시켰다.
- 공기역학적 발열 제어: 대용량 데이터 처리 시 발생하는 쓰로틀링을 방지하고자 하단 타공 설계를 적용해 열 안정성을 확보하였다.
- 안정적 전원 분배 네트워크: GMSL 캡처 보드와 Jetson의 전압 강하를 막기 위해 DC-DC 레귤레이터를 전단에 배치하여 딥러닝 추론 시 정전압을 유지하도록 회로를 설계하였다.



4.3 소프트웨어 아키텍처 및 흐름도

소프트웨어는 센서 드라이버 노드, 딥러닝 PCD 전처리 노드, 위치 추정 및 맵핑 노드, 스케일 정합 노드로 나뉘어 다중 스레드 환경에서 병렬 실행된다.

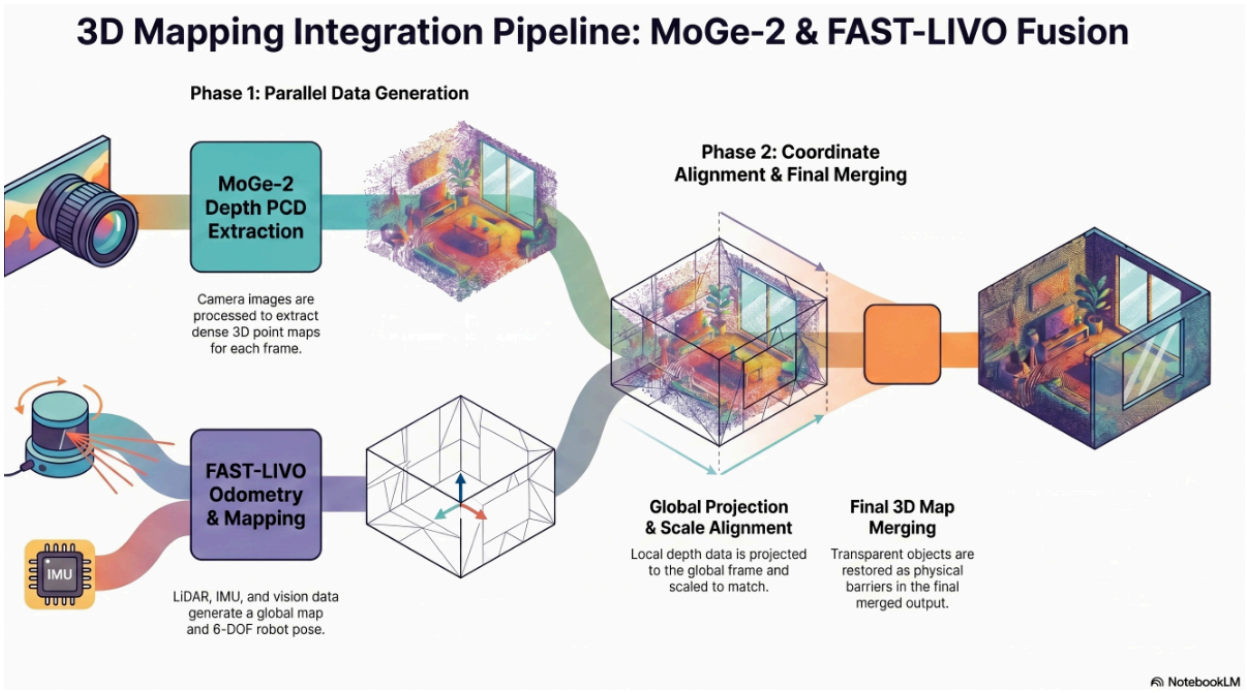
1. Sensor Interface Node: 하드웨어 트리거된 센서 데이터를 수신하고 ROS 메시지 표준 규격으로 변환한다.

2.Deep Learning Pre-processing: GPU 추론을 통해 MoGe-2 네트워크를 구동하여 2D 이미지로부터 직접 3D Depth PCD를 추출해 낸다.

2. FAST-LIVO Node: LiDAR, IMU, Camera 피처를 융합하여 로봇의 실시간 Odometry를 계산하고 전역 맵(Global Map)을 생성한다.

3. Scale Alignment: 완성된 FAST-LIVO 맵과 MoGe-2 PCD 간의 최적화된 스케일 및 좌표축 변환 연산을 수행한다.

4.4 전체 시스템 데이터 흐름도 **(System Flowchart)**



1. 데이터 취득 및 **MoGe-2** 전처리 **(Pre-processing):** 동기화된 카메라 이미지가 MoGe-2 모델에 입력되어 프레임별로 밀도 높은 3D 포인트 맵(Depth PCD)이 추출된다.

2. FAST-LIVO 매핑 및 오도메트리 산출 **(Odometry & Mapping):** 동시에 라이다, IMU, 비전 데이터가 FAST-LIVO 프레임워크에 입력되어 점-대-평면(Point-to-plane) 정합 기반의 글로벌맵이 작성되며, 각 프레임마다의 로봇 6자유도 자세(Odometry)가 산출된다.

3. 오도메트리 주입 **(Odometry Injection):** 카메라 좌표계(Local Frame)를 기준으로 생성된 MoGe-2의 Depth PCD 데이터에, FAST-LIVO가 산출한 해당 시점의 로봇 Odometry를 주입하여 전역 좌표계(Global Frame)로 투영한다.

4. 기존 맵과의 스케일 정합 **(Scale Alignment):** 투영된 Depth PCD를 기존 FAST-LIVO 글로벌 맵의 포인트들과 비교하여 스케일(Scale) 및 세부 병진(Translation) 오차를 최소화하는 정합 알고리즘(예: Kabsch-Umeyama)을 적용한다

5. 최종 **3D** 맵 병합 **(Map Merging):** 스케일이 완벽히 일치된 Depth PCD를 기존 맵에 병합함으로써, 라이다가 인식하지 못했던 유리창 등의 투명 객체가 물리적인 장벽으로 복원된 최종 지도를 출력한다.

5. 구현 내용

5.1.1 현재까지 구현한 내용

1. 다중 센서 데이터 동기화 및 전처리 파이프라인 구축

실제 로봇 주행 환경에서 취득되는 이종 센서(Livox 라이다, ZED X 스테레오 카메라, 고정밀 IMU)의 데이터는 각기 다른 샘플링 주파수와 타임스탬프를 갖는다. 이를 해결하기 위해 ROS 2 기반의 다중 스레드 데이터 수신 및 동기화 버퍼를 구현하였다.

라이다 스캔의 시작/종료 시간을 기준으로 큐(Queue)에 적재된 IMU와 이미지 데이터를 정렬하였다.

카메라 영상 처리 지연을 고려하여 타임스탬프 오프셋을 적용하고, 비정상적인 데이터 루프백이나 타임 점프 예외 처리 로직을 구현하여 시스템의 안정성을 확보했다.

2. VoxelMap 기반 실시간 상태 추정 및 맵 관리 **(LIO/VIO)**

FAST-LIVO 프레임워크를 기반으로 실시간 6자유도(6-DoF) 오도메트리 산출 및 로컬 맵 관리를 구현하였다.

voxel_map.cpp 내에 옥트리(Octree) 기반 복셀 맵을 구성하고, 점군의 공분산 행렬에 대한 고유값 분해(Eigen Decomposition)를 통해 기하학적 평면을 추출하였다. 추출된 평면의 법선 벡터(Normal Vector)를 활용하여 상태 추정기(Iterated Kalman Filter)의 잔차(Residual)를 최소화하는 방향으로 로봇의 자세를 갱신한다.

엣지 컴퓨팅 환경의 한정된 메모리를 효율적으로 사용하기 위해, 로봇의 현재 위치를 중심으로 일정 반경을 벗어난 복셀 맵 데이터를 메모리에서 동적으로 해제하는 슬라이딩 윈도우 기법을 적용하였다.

3. 시각적 **3D** 복원 데이터**(Depth PCD)** 정합 및 스케일 최적화

라이다가 인식하지 못하는 투명 객체(유리창 등) 영역을 복원하기 위해, 비전 모델에서 생성된 Depth 점군을 기존 라이다 전역 지도에 융합하는 모듈을 구현하였다.

```
if (depth_integration_en) {
  // Apply 90 degree Pitch UP rotation (around Y axis)
  // to correct the reported 90 degree downward tilt.
  double angle = -90.0 * M_PI / 180.0; // Negative 90 on Y often lifts the front up
epending on convention
  Eigen::Matrix3d R_pitch_up;
  R_pitch_up << cos(angle),  0, sin(angle),
                0,           1, 0,
                -sin(angle), 0, cos(angle);
  // 시선 축 기준으로 90도 회전 (Roll 보정)
/ Z축 회전 적용
  double roll_angle = 90.0 * M_PI / 180.0;
  Eigen::Matrix3d R_roll;
  R_roll << 1,          0,               0,
            0, cos(roll_angle), -sin(roll_angle),
            0, sin(roll_angle),  cos(roll_angle);

  R_c2i = R_roll*R_pitch_up;
  T_c2i = Eigen::Vector3d::Zero();

  loadDepthFileIndex();
```

<Depth Image 변환 행렬>

카메라와 IMU 간의 설치 각도 차이로 인한 시선 축 오차를 보정하기 위해 Pitch 및 Roll 회전 변환 행렬을 적용한 후 해당 프레임의 과거 로봇 전역 자세를 조회하여 지역 좌표계의 Depth 점군을 전역 공간으로 투영시켰다.

```
        // Step 1: Compute optimal scale for THIS specific frame
        double sum_num = 0.0, sum_den = 0.0;
        int match_count = 0;

        for (size_t j = 0; j < cloud->points.size(); j += 50) { // Sample points for speed
            const auto &pt = cloud->points[j];
            Eigen::Vector3d p_c(pt.x, pt.y, pt.z);
            double dist = p_c.norm();
            if (dist > depth_max_dist || dist < 0.5) continue; // Distance filter

            Eigen::Vector3d p_w_base = pose.rot_end * (R_c2i * p_c + T_c2i) + pose.pos_end; // s=1.0

            float vs = voxelmap_manager->config_setting_.max_voxel_size_;
            VOXEL_LOCATION pos((int64_t)(p_w_base.x()/vs), (int64_t)(p_w_base.y()/vs), (int64_t)(p_w_base.z()/vs));

            auto it = voxelmap_manager->voxel_map_.find(pos);
            if (it != voxelmap_manager->voxel_map_.end() && it->second->plane_ptr_->is_plane_) {
                auto& plane = it->second->plane_ptr_;

                // Distance-based weighting (WLS)
                double weight = 1.0 / (dist * dist + 1e-4);

                double den = plane->normal_.dot(pose.rot_end * R_c2i * p_c);
                double num = -(plane->normal_.dot(pose.rot_end * T_c2i + pose.pos_end) + plane->d_);
                if (std::abs(den) > 1e-4) {
                    sum_num += weight * num * den;
                    sum_den += weight * den * den;
                    match_count++;
                }
```

비전 기반 점군의 본질적인 아웃라이어를 제거하기 위해 라이다 복셀 맵의 확정적인 평면과 거리가 너무 먼 점군을 배제하였다.

```

        // Step 2: Accumulate points into a grid for averaging and noise filtering
        for (auto &pt : cloud->points) {
            Eigen::Vector3d p_c(pt.x * local_scale, pt.y * local_scale, pt.z * local_scale);
            if (p_c.norm() > depth_max_dist * local_scale || p_c.norm() < 0.5) continue; // Distance filter

            Eigen::Vector3d p_w = pose_it->second.rot_end * (R_c2i * p_c + T_c2i) + pose_it->second.pos_end;

            float fs = depth_filter_size;
            VOXEL_LOCATION d_pos((int64_t)(p_w.x()/fs), (int64_t)(p_w.y()/fs), (int64_t)(p_w.z()/fs));
            // Correct indexing for negative values
            if (p_w.x() < 0) d_pos.x -= 1;
            if (p_w.y() < 0) d_pos.y -= 1;
            if (p_w.z() < 0) d_pos.z -= 1;

            auto& cell = final_voxel_grid[d_pos];
            cell.pos += p_w;
            cell.color += Eigen::Vector3d(pt.r, pt.g, pt.b);
            cell.count++;
        }

```

또한, 복셀 그리드 내에 누적된 포인트의 관측 횟수를 카운팅하여, 최소 3회 이상 중복 관측된 신뢰도 높은 점군만을 유효한 물리 장벽으로 인정하는 필터링을 적용했다.

```
    // Step 3: Final Fusion and Noise Removal
    all_depth_cloud->clear();
    for (auto const& [pos, data] : final_voxel_grid) {
        // Noise Filter: Only keep voxels seen in at least 3 points/frames
        // This removes random floating noise and single-frame depth errors
        if (data.count < 3) continue;

        pcl::PointXYZRGB averaged_pt;
        Eigen::Vector3d final_pos = data.pos / data.count;
        Eigen::Vector3d final_col = data.color / data.count;

        averaged_pt.x = final_pos.x();
        averaged_pt.y = final_pos.y();
        averaged_pt.z = final_pos.z();
        averaged_pt.r = static_cast<uint8_t>(final_col.x());
        averaged_pt.g = static_cast<uint8_t>(final_col.y());
        averaged_pt.b = static_cast<uint8_t>(final_col.z());

        all_depth_cloud->push_back(averaged_pt);
    }

    std::string refined_map_dir = std::string(ROOT_DIR) +
"Log/PCD/dense_refined_final.pcd";
    pcl::PCDWriter pcd_writer;
    pcd_writer.writeBinary(refined_map_dir, *all_depth_cloud);
    std::cout << BOLDGREEN << "[ Refinement ] Done! Cleaned map saved to: " <<
```

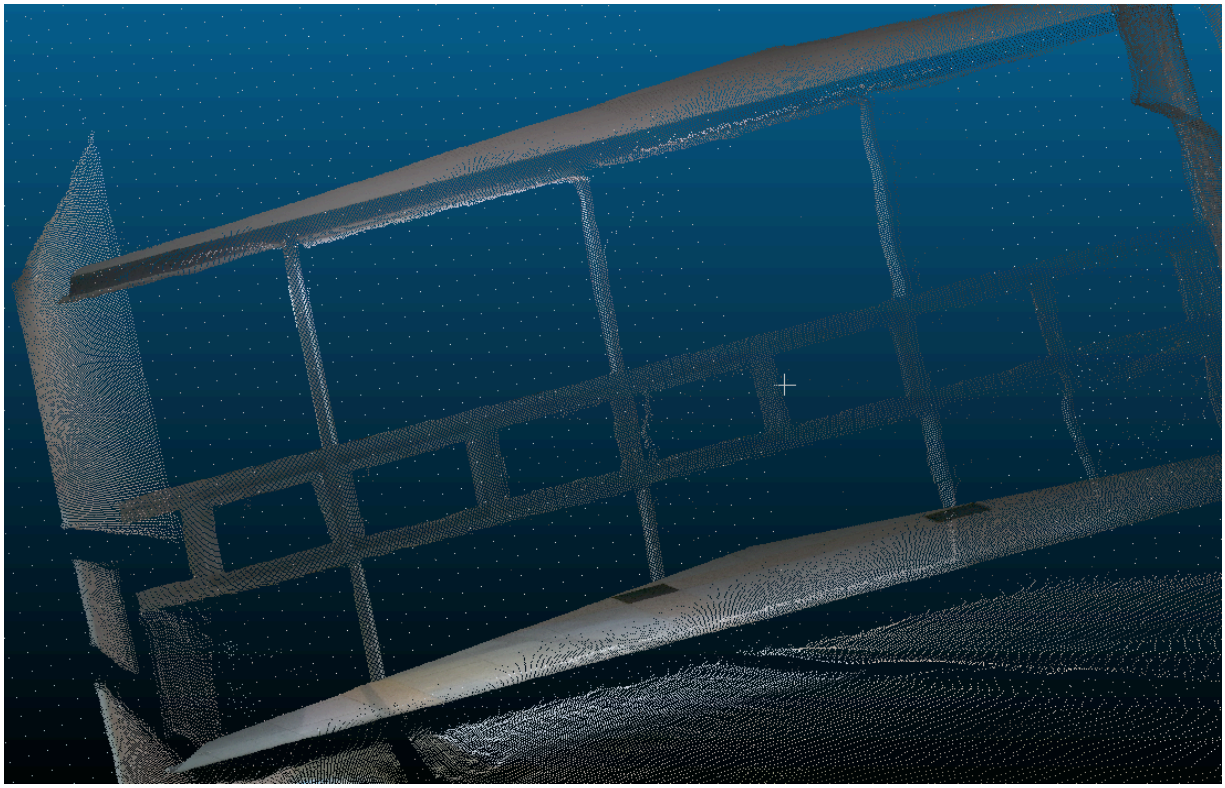
모듈 내에서 단안 뎁스의 스케일 모호성(Scale Ambiguity)을 해결하기 위한 후처리 정합 로직을 구현하였다.

기존 맵의 평면 법선 벡터와 투영된 점군 간의 거리에 기반한 가중치를 두어,각 프레임별 최적의 스케일 팩터를 해석적으로 산출하여 정밀도를 높였다.

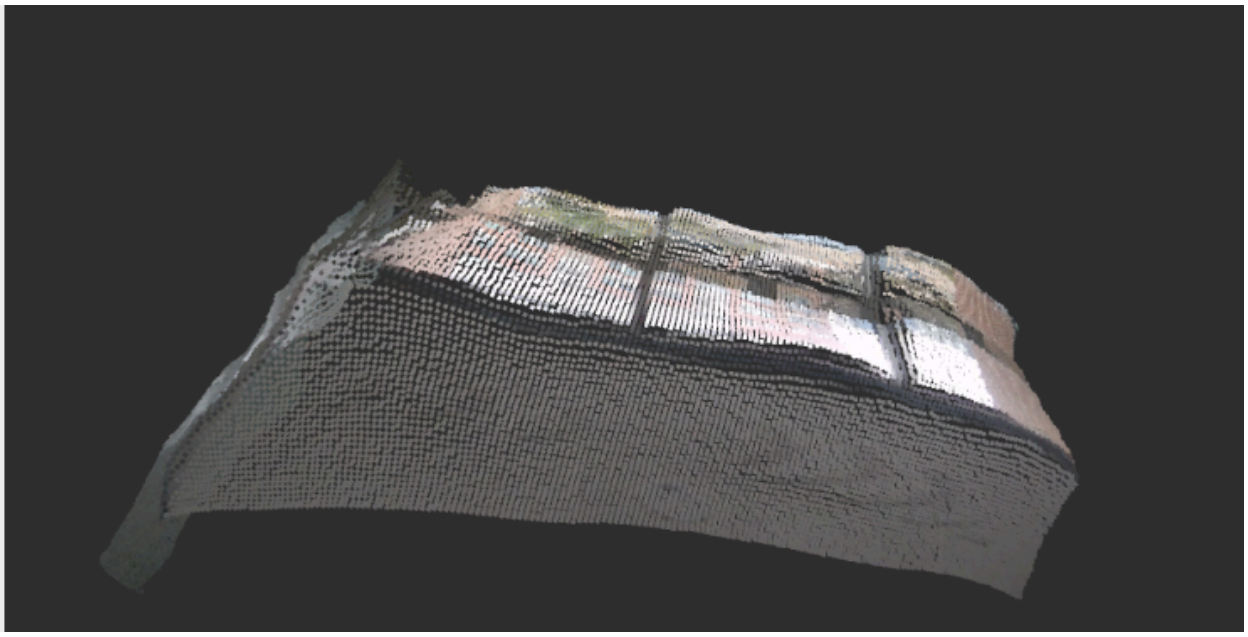
5.2 제작 과정 및 주요 설계 변경 사항

본 시스템은 초기 설계 단계부터 최종 구현에 이르기까지, 투명 객체(유리창) 인식이라는 핵심 목표를 달성하기 위해 다양한 알고리즘을 테스트하고 아키텍처를 반복적으로 수정하는 과정을 거쳤다.

초기 매핑 프레임워크로는 연산 효율성이 뛰어난 라이다-관성 기반의 FAST-LIO를 도입하였다 [1]. 그러나 라이다 센서의 신호가 투과되어 버리는 투명 객체의 본질적인 한계를 극복하기 위해서는 시각적 맥락(Visual Context)의 융합이 필수적이라고 판단하여, 비전 데이터 통합 처리가 가능한 FAST-LIVO 프레임워크로 시스템 아키텍처를 전면 개편하였다 [2].



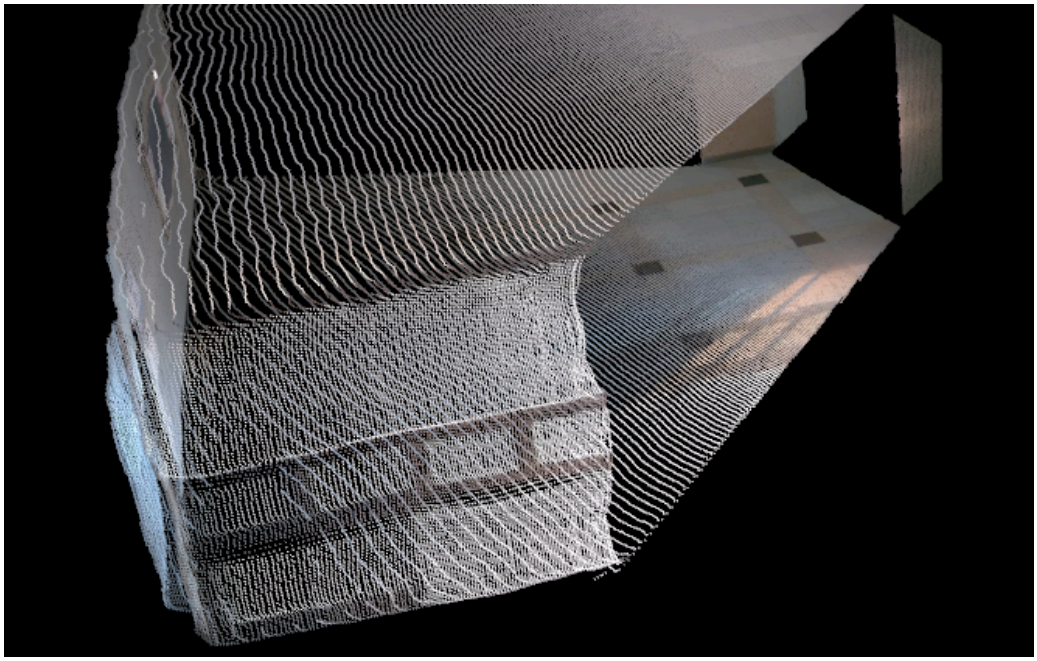
<fast-foundation stereo>



<depth anything V3>

가장 많은 설계 변경이 발생한 부분은 2D 이미지로부터 3D 구조를 복원하는 뎁스 추론 모델의 선정 과정이었다. 초기에는 연산 속도 확보를 위해 Fast Foundation Stereo 모델을

적용하였으나, 스테레오 매칭 방식의 특성상 텍스처가 없는 투명 유리창 영역에서는 유의미한 뎁스(Depth) 특징점을 추출하지 못하는 결함이 확인되었다. 이를 해결하고자 단안 뎁스 추정의 최신 모델인 Depth-Anything-V3로 교체하여 테스트를 진행하였다. 해당 모델은 유리창 객체 자체를 인식하고 뎁스 맵을 분리해 내는 데는 성공하였으나, 이를 3D 포인트 클라우드로 변환했을 때 기하학적 3D 구조가 심하게 붕괴되어 로봇의 실제 장애물 회피용 지도로는 활용할 수 없었다.


<moge-2>

이러한 실패 원인 분석을 거쳐, 최종적으로 장면의 기하학적 구조를 아핀 불변 및 메트릭 스케일로 정밀하게 예측하는 MoGe-2 모델을 최종 채택하였다. MoGe-2를 도입한 결과, 유리창의 존재 유무뿐만 아니라 그 표면을 기하학적 왜곡 없이 고밀도 3D 평면으로 완벽히 복원할 수 있었으며, 이는 결과적으로 전체 맵핑 시스템의 신뢰도를 대폭 향상시키는 결정적인 설계 변경이 되었다.

6. 실험 및 결과

6.1 검증 방법 및 평가 기준

대학 건물 내 도서관 및 2층 복도에서 취득한 ROS Bag 데이터를 활용하여, 다음 두 가지 기준으로 알고리즘을 평가하였다.

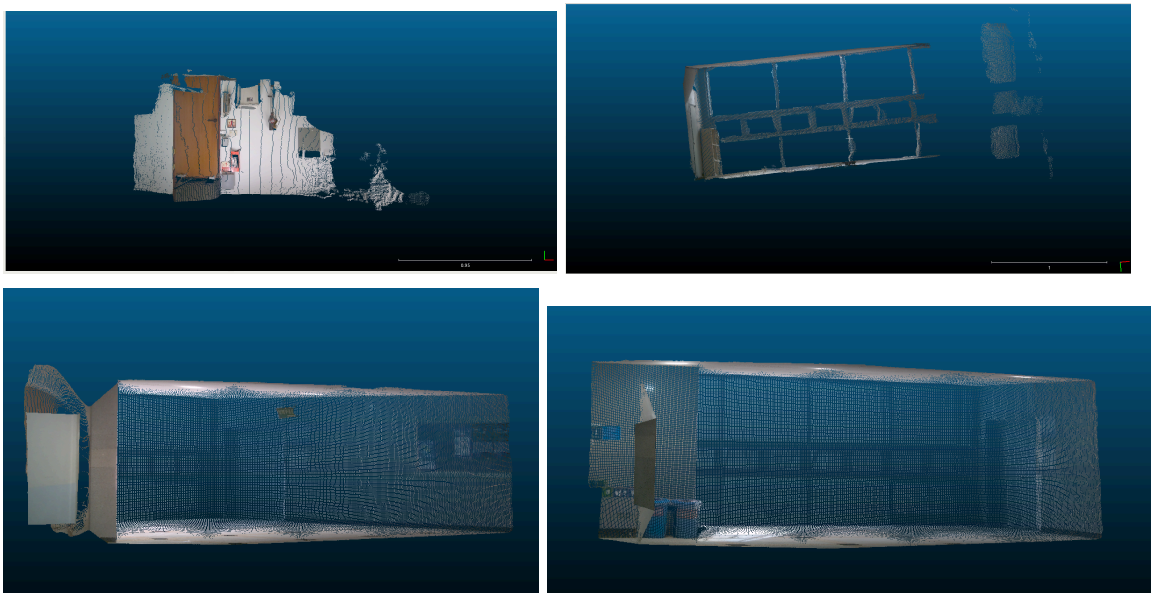
1. 일반 구조물의 뎁스 복원력: 텍스처가 부족한 하얀 벽이나 매끄러운 바닥에서 결측치 없이 조밀하게 맵이 생성되는가?
2. 투명 물체의 표면 인식력: 유리창 투과 현상을 억제하고 순수 유리 표면에 뎁스가 맺히는가?

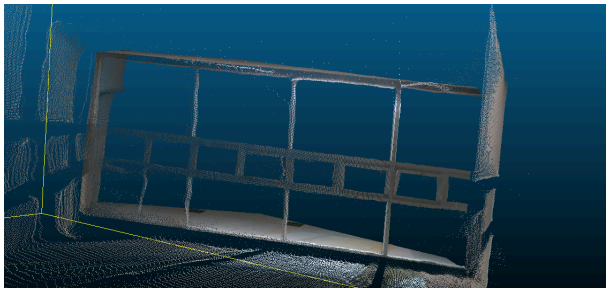
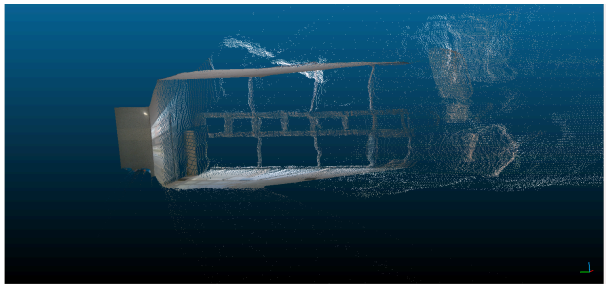
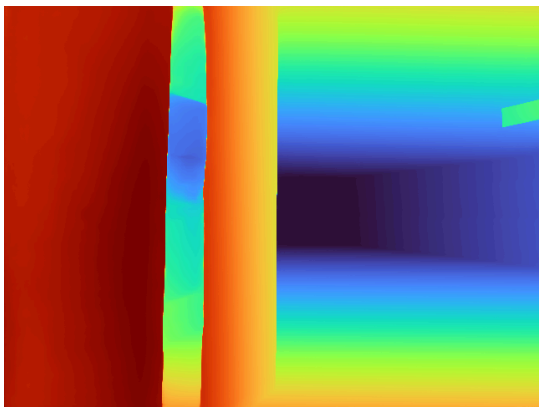
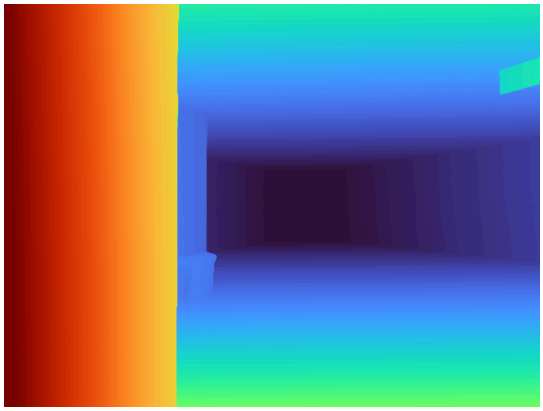
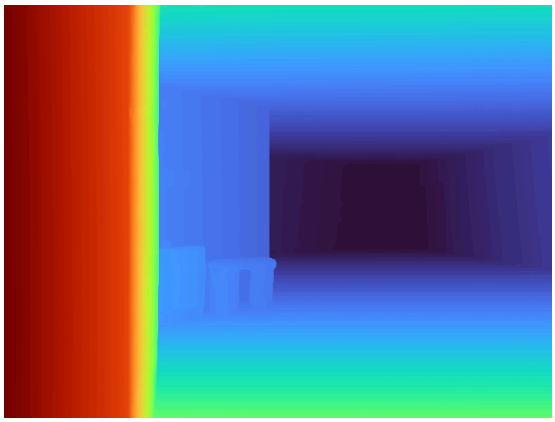
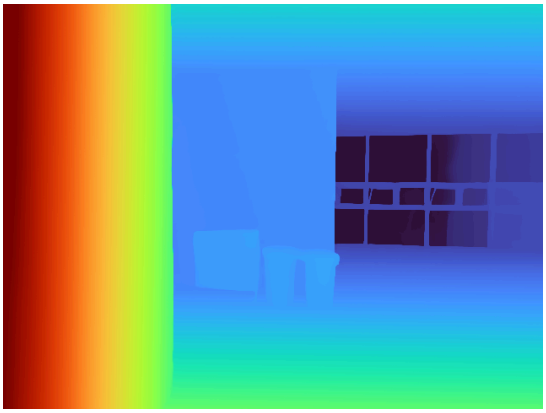
6.2 알고리즘 성능 비교 **(**실험 결과**)**

본 프로젝트에서는 적용한 뎁스 추정 알고리즘의 문제점을 기하학적·시각적으로 분석하고 이를 보완할 수 있는 다음 단계의 알고리즘으로 선회하는 과정을 반복하였다. 이 과정에서 수행한 알고리즘들의 적용 결과 및 성능 비교는 다음과 같다.

1. **Fast-FoundationStereo** 및 **FoundationStereo (**기각**)**

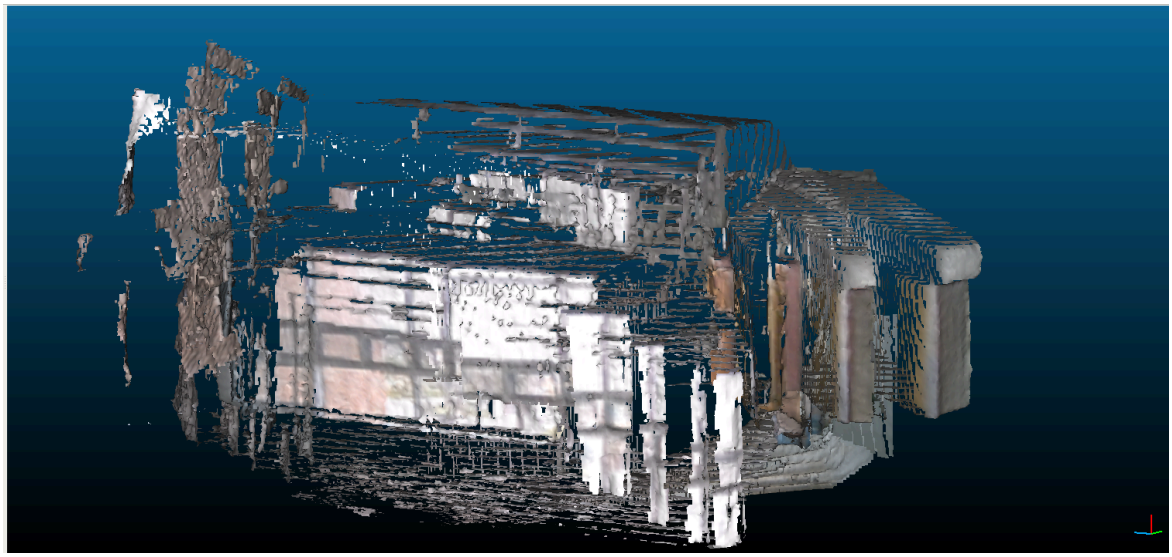
   NVIDIA의 스테레오 모델을 적용한 결과, 사선 방향에서 형광등 반사광을 단서로 찰나의 순간 유리를 인식하는 데는 성공하였다. 그러나 정면에서는 뎁스를 추정하지 못해 유리 영역이 뚫려버렸으며, 특히 텍스처가 없는 벽면에서 시차 매칭에 실패해 건물 뼈대 맵이 붕괴되는 현상이 발생하여 기각하였다.

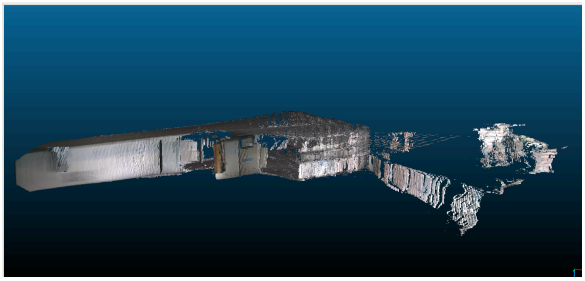


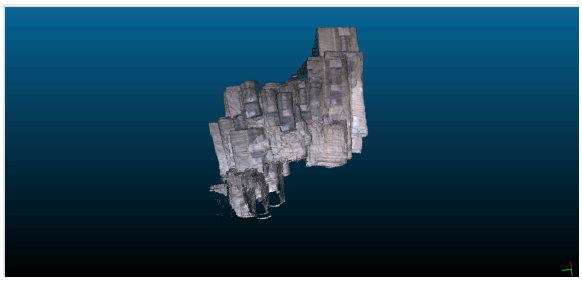







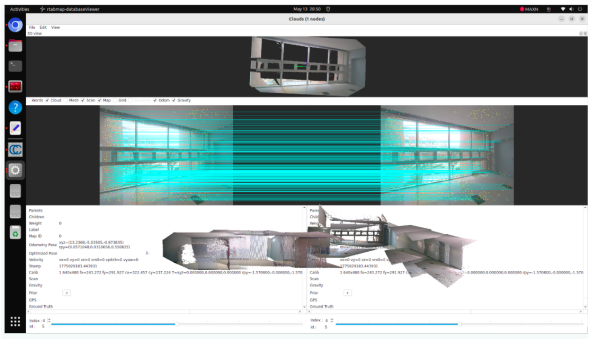
2. **TSDF** 및 **RTAB-Map** 융합 시도 **(**기각**)**

찰나의 인식 결과를 3D 복셀에 누적하여 표면을 굳히기 위해 TSDF 알고리즘과 RTAB-Map 연동을 시도하였다. 그러나 SLAM 노드와 AI 노드 간의 데이터 처리 지연으로 시간 동기화가 어긋나 타임머신 현상이 발생하였고, 결국 궤적 소실 및 맵 붕괴를 초래하였다.





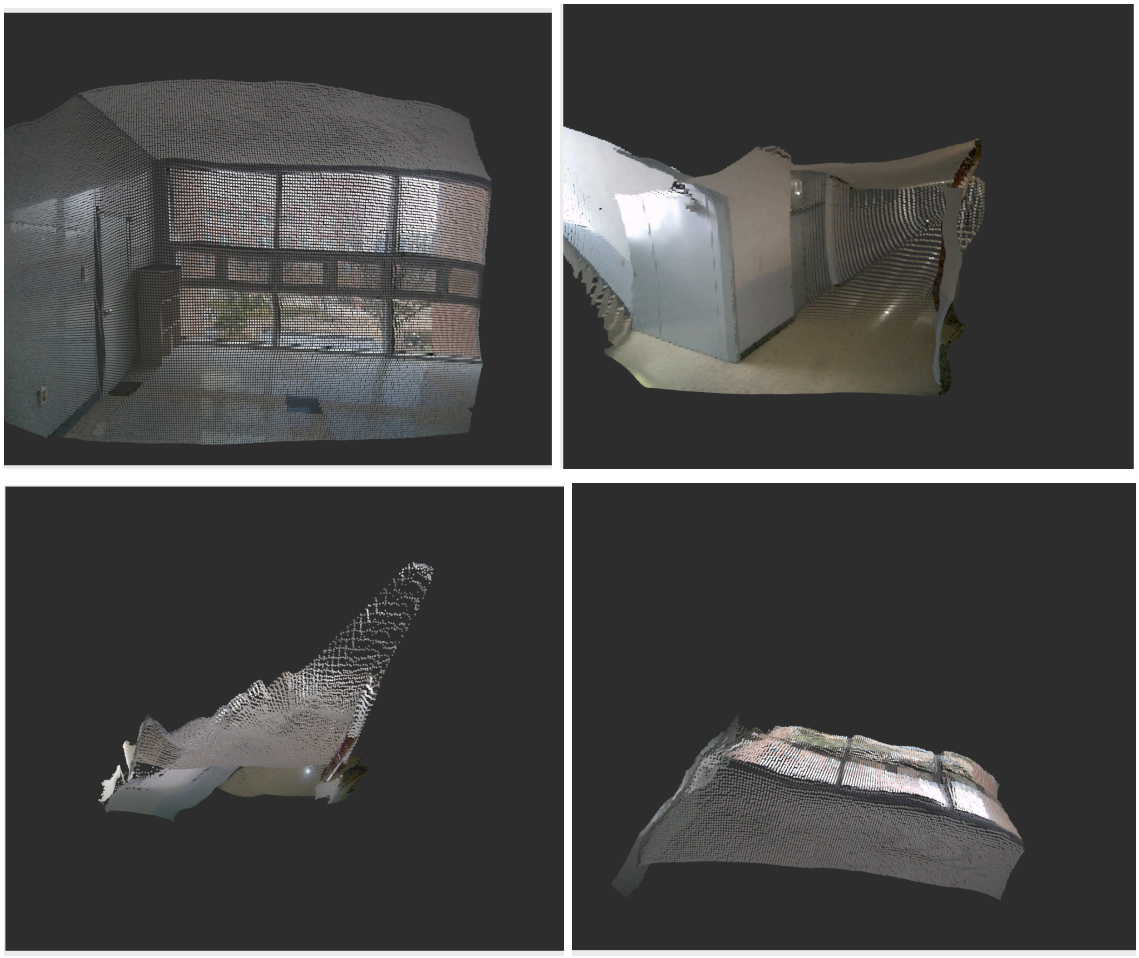






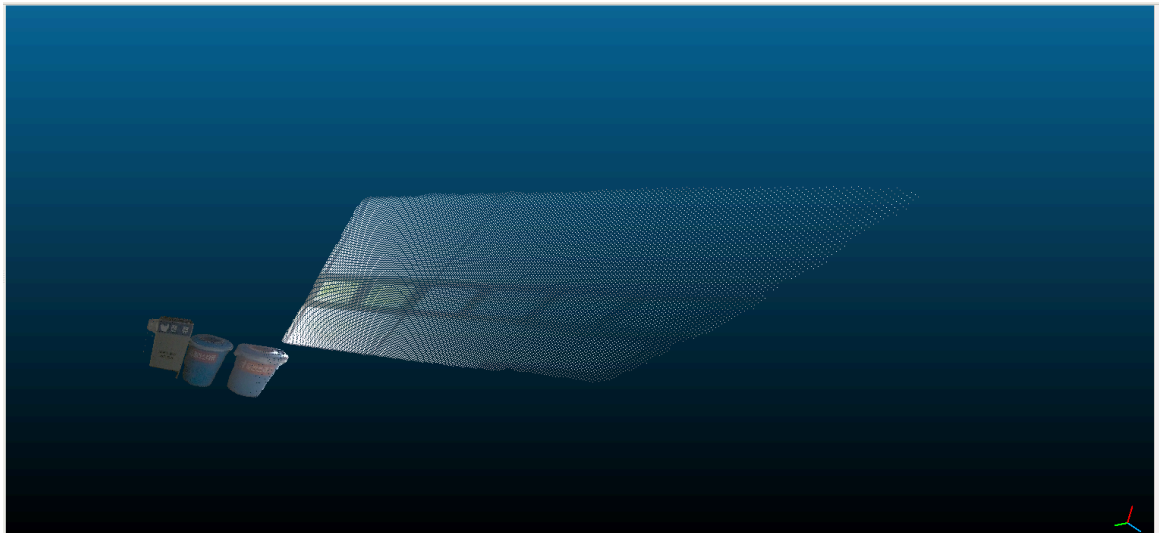
3. **Depth-Anything** 기반 뎁스 추정 **(**기각**)**

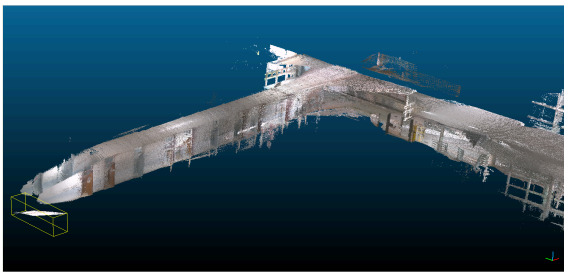
결측치 문제는 해결되었으나, 핀홀 카메라 모델에 상대 뎁스 값을 대입한 결과, Z값이 커질수록 X, Y 좌표가 비례하여 팽창하는 심각한 터널 형태의 스케일 팽창 왜곡이 발생하였다.

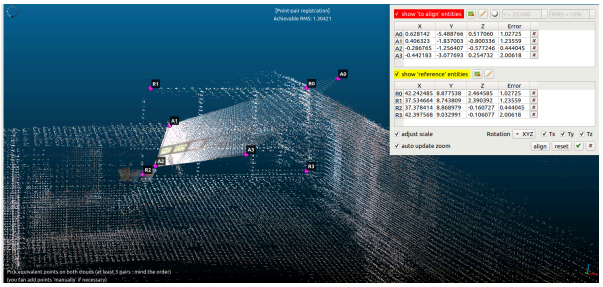


4. **MoGe-2** 및 **Grounded-SAM 2** 기반 오프라인 융합 시도

   절대 뎁스의 스케일 문제를 해결하기 위해 MoGe-2를 도입하고, 유리를 투과하는 현상을 막기 위해 Grounded-SAM 2를 활용해 아래 그림과 같이 유리 마스크와 뎁스를 성공적으로 **5) MoGe-2** 및 **Grounded-SAM 2** 기반 오프라인 융합







그러나 파이썬에서 조작된 점군을 상용 3D 툴(CloudCompare)로 병합하려 했을 때, 툴의 강체 변환 한계로 인해 사다리꼴 왜곡을 직사각형 창틀 규격에 완벽히 펴서 맞추는 것에 실패하였다.

5. **MoGe-2** 절대 뎁스 추출 및 **LiDAR** 최적화 융합 **(**현재 파이프라인**)**

   오프라인 툴의 정렬 한계를 극복하기 위해, MoGe-2의 데이터를 FAST-LIVO2의 C++ 백엔드 로직 내부로 직접 통합하는 구조로 전면 개편하였다. 시스템 과부하를 막기 위해 아래와 같이 3단계 아키텍처를 도입하였다.

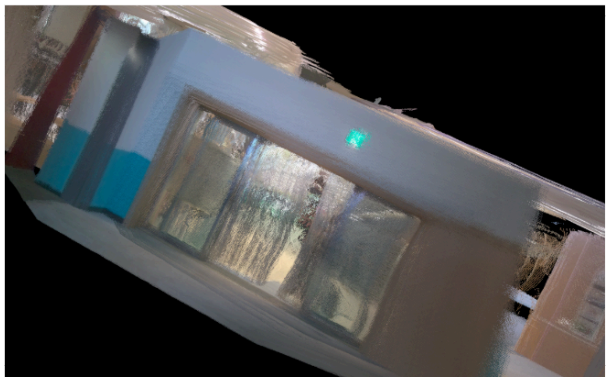
- Phase 1 (실시간 뼈대 생성): 라이다와 IMU 기반으로 실시간 궤적 및 뼈대 생성.
- Phase 2 (오프라인 1차 융합): SLAM 종료 직후 카메라 궤적을 기반으로 MoGe-2 뎁스를 글로벌 투영.
- Phase 3 (라이다 최적화): 분산 기반 가중치 평균을 도입하여, 투영된 점들이 라이다 평면과 멀면 버리고 신뢰할 수 있는 점들만 라이다 평면에 달라붙게 깎아냄.

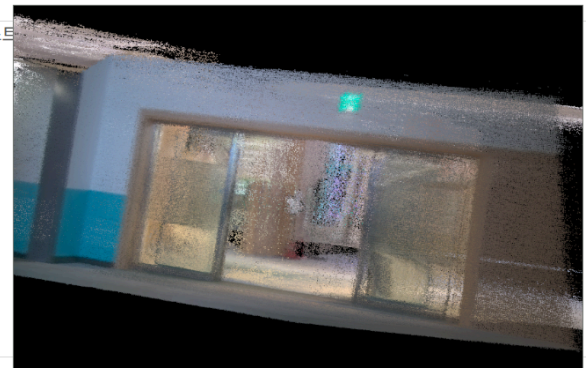
이 과정을 거쳐 단순 병합을 넘어선 정밀한 최적화 3D 맵핑에 성공하였다.



Depth merged

Optimized depth map



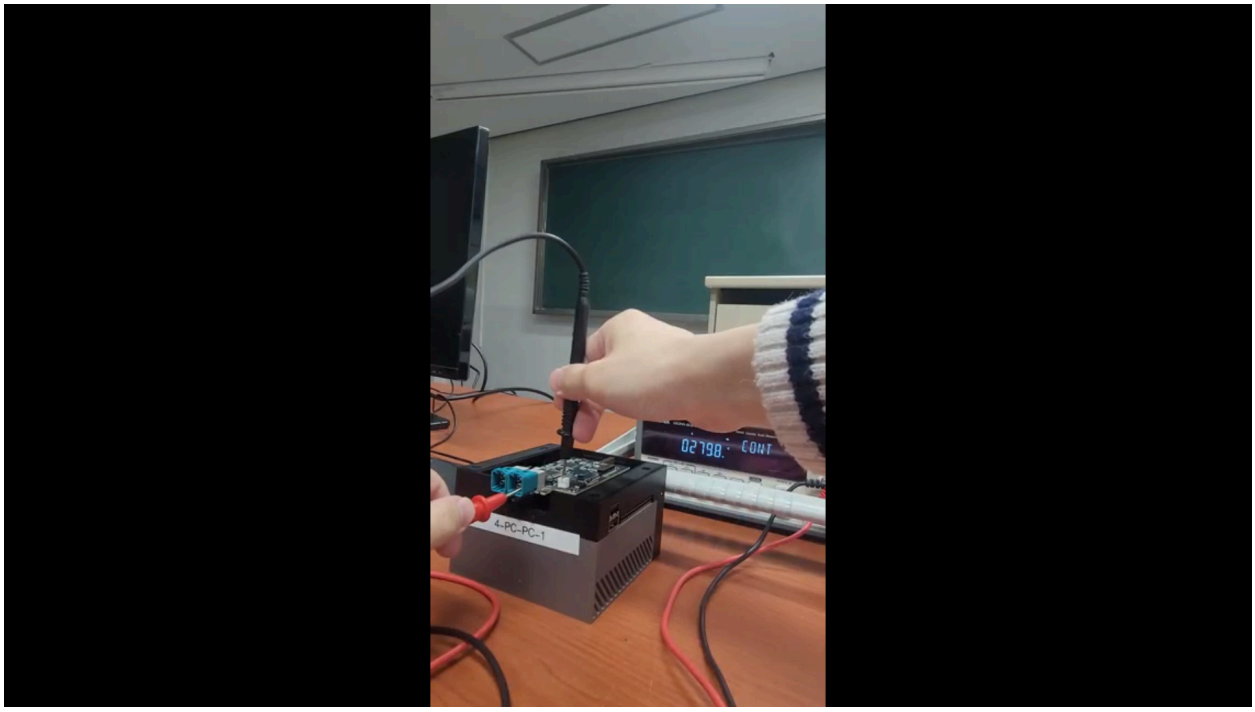


6.3 기타 실패 사례 및 문제점 분석 **(**트러블슈팅**)**

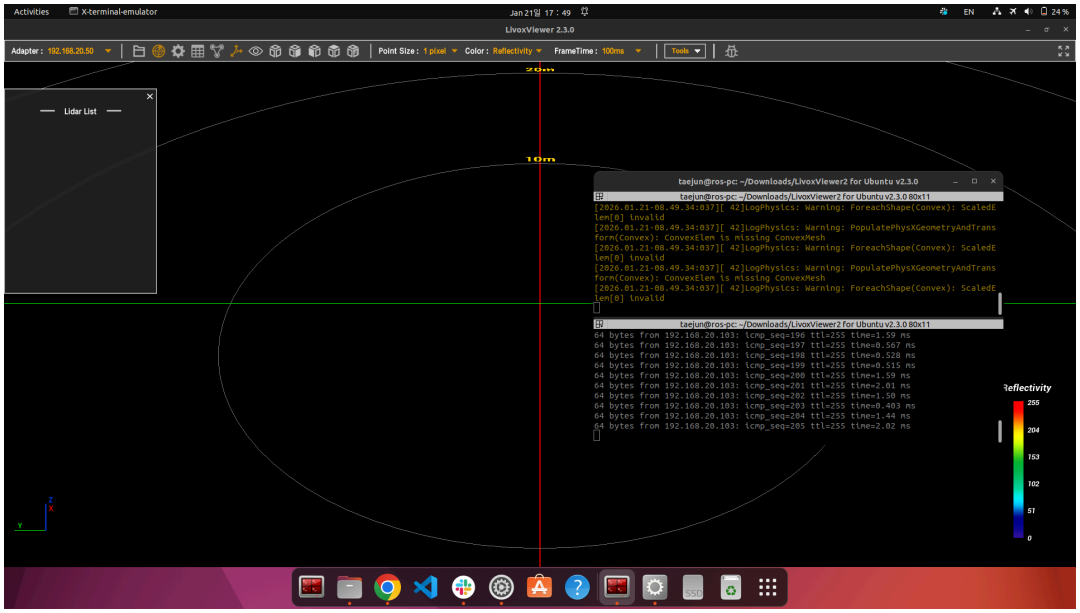
본 프로젝트는 완성된 패키지를 단순히 실행하는 것을 넘어, 하드웨어 셋업부터 커스텀 센서 통합, 엣지 컴퓨팅 환경 구축, 알고리즘 융합에 이르기까지 시스템 전체를 통합하였다. 이 과정에서 발생한 주요 문제점과 해결 과정은 다음과 같다.

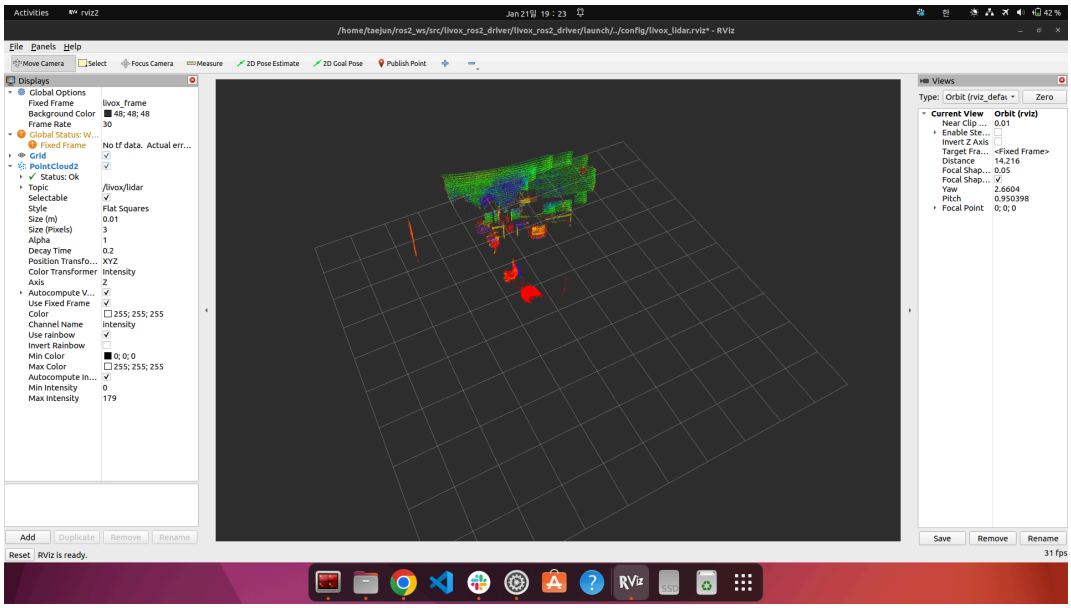
1. 하드웨어 및 센서 통합 이슈

- JST 케이블 극성 역전: ZED 캡쳐보드 전원 연결 시 산업 표준 부재로 역전압이 발생하였으나, 멀티미터 통전 테스트를 실시하여 정확한 극성 파악 후 재납땜하여 방어하였다. 테스트 영상: https://drive.google.com/file/d/19i7SFnMaUz9v1zjATzG6B57PuJeOZH42/view?usp=sharing)

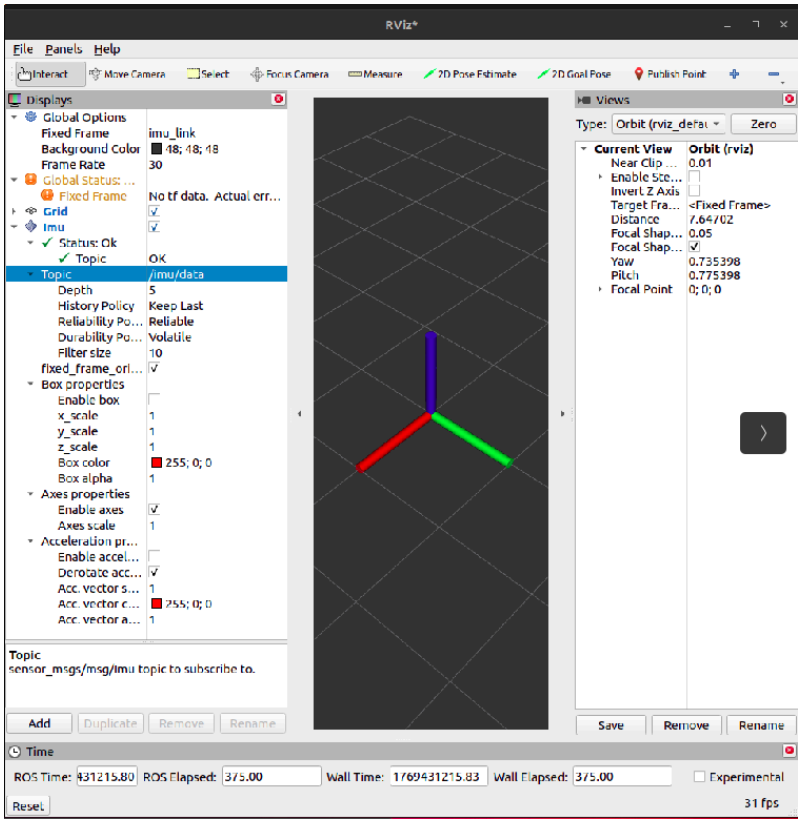


- 라이다 IP 추적 및 통신 확보: 초기 Livox LiDAR 연결 시 공식 매뉴얼의 기본 IP 통신이 불가능한 문제가 발생하였다. tcpdump를 이용한 패킷 스니핑을 통해 센서의 내부 할당 IP를 역추적하였고, 호스트 PC의 서브넷을 수동으로 동기화하여 통신을 확보하였다. (라이다 디버깅 상세 로그: https://github.com/taejunKim-GJ/Enviroment-Mapping-Slam/issues/1)



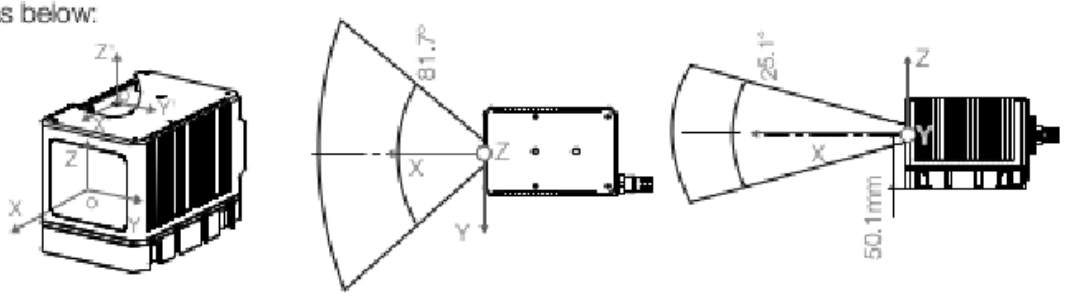


- 내장 IMU 아키텍처 재설계: 외부 IMU의 패킷을 직접 파싱하다가, 시간 동기화 오차를 근본적으로 해결하기 위해 라이다 내장 IMU를 사용하는 것으로 하드웨어 구조를 리팩토링하였다. (IMU 디버깅 상세 로그: https://github.com/taejunKim-GJ/Enviroment-Mapping-Slam/issues/2)



Coordinates

The Horizon has a built-in IMU. The coordinates of the point cloud O-XYZ and of the IMU O'-X'Y'Z' are defined as below:



The origin O' of IMU coordinates is defined in the point cloud coordinates as (-55.12, -22.26, 29.70) (Unit: mm).

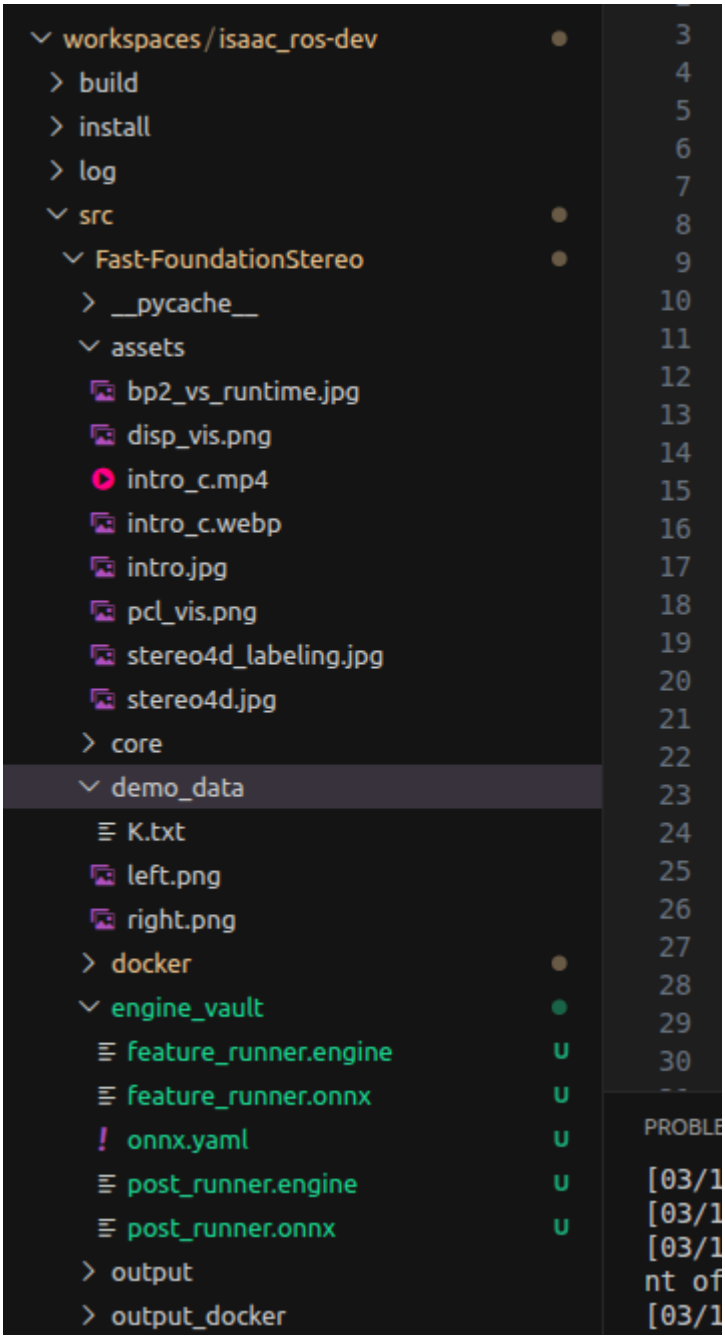
Figure 5.1.1 Coordinates of the Horizon

2. 소프트웨어 빌드 및 **Edge** 환경 구축

- ROS 2 드라이버 C++ 호환성: GCC 11 이상에서 발생하는 헤더 누락 및 링커 에러를 소스 수정 및 공유 라이브러리 옵션 추가로 해결하였다.

- ARM64 환경 TensorRT 엔진 최적화: Jetson 환경의 아키텍처 충돌 에러를 우회하고, 호환되지 않는 OpenAI Triton을 PyTorch 연산으로 대체하였다. 또한 C++ 기반 순정

컴파일러를 직접 호출해 2시간에 걸친 오프라인 엔진 빌드를 수행하여 엣지 추론 환경을 완성했다.



7. 결론 및 향후 계획

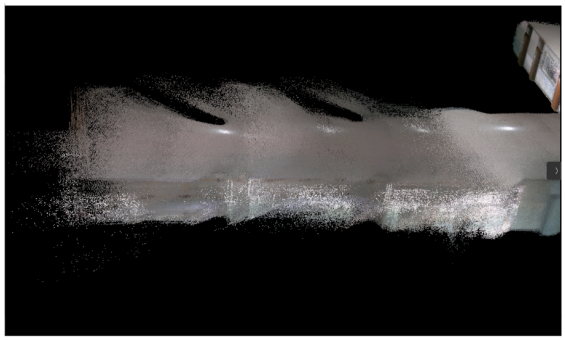
7.1 1학기 진행 내용 요약

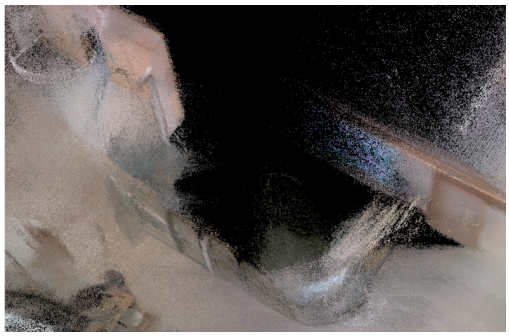
본 프로젝트는 투명 물체가 다수 존재하는 실내 환경에서 기존 LiDAR-Camera 기반 SLAM이 가지는 매핑 한계를 극복하기 위해 진행되었다. 이를 해결하기 위해 다양한 뎁스 추정 모델의 기하학적 한계를 교차 분석하였으며, 최종적으로 MoGe-2의 절대 뎁스가 실내 3D 복원에 가장 적합함을 확인하였다.

특히, MoGe-2의 뎁스를 단순히 병합하는 오프라인 툴의 한계를 극복하고자, FAST-LIVO2의 C++ 백엔드에 직접 개입하여 LiDAR 뼈대 맵을 평면 기준으로 삼고 카메라 뎁스의 노이즈를 깎아내는 최적화 융합 파이프라인을 자체적으로 구축함으로써 밀도 높은 1차 3D 맵 생성에 성공하였다.

7.2 현재 한계점

그러나 최적화를 거친 1차 맵에서도, 해결해야 할 두 가지 기하학적 한계점이 남아있다.





- 투명 매질의 기하학적 투과 현상**:** 단안 비전(MoGe-2)의 특성상 유리를 투명한 허공으로 인식하여, 실제 벽면(유리)이 아닌 유리창 너머의 내부 객체(도서관 책장 등)가 맵에 함께 투영되는 현상이 발생한다.
- 원통형 유리의 곡률 피팅 왜곡**:** FAST-LIVO2의 기존 최적화 알고리즘은 노이즈 점군을 '단일 평면'으로 억지로 깎아내려는 특성이 있어, 곡면을 가진 투명 유리의 경우 수학적 피팅 로직과 충돌하며 맵이 기하학적으로 완전히 깨지는 현상이 발생한다.

7.3 향후 개선 계획 **(2**학기 핵심 과제**)**

본 프로젝트의 최종 목표인 "유리창 너머의 물체가 배제되고 유리면만 단단한 벽처럼 인식되게 하는 것"을 달성하기 위해, 차학기에는 다음 두 가지 핵심 과제를 수행할 계획이다.

- 통제된 환경에서의 데이터 고도화: 맵의 정합성과 스케일 안정성을 높이기 위해 차기 데이터 취득 시, 폐루프를 완벽히 구성하고 동적 객체의 개입을 최소화할 것이다.

- 시맨틱 세그멘테이션 기반 2차 맵 정제 완성: 현재 구축된 1차 맵 위에 Grounded-SAM 2 기반의 마스크를 3D 공간으로 투영시키는 2차 정제 파이프라인을 결합할 것이다. 마스크 테두리의 뎁스를 유리 내부로 덮어씌워 투과 현상을 차단하고 곡면 구조를 복원하여, 최종적으로 무결점의 3D 가우시안 스플래팅 환경 모델을 렌더링하고자 한다.

8. 참고문헌

[1] hku-mars, "FAST_LIO," GitHub Repository, 2021. [Online]. Available: https://github.com/hku-mars/FAST_LIO

[2] hku-mars, "FAST-LIVO2," GitHub Repository, 2024. [Online]. Available: https://github.com/hku-mars/FAST-LIVO2

[3] NVlabs, "Fast-FoundationStereo," GitHub Repository, 2026. [Online]. Available: https://github.com/NVlabs/Fast-FoundationStereo

[4] B. Wen, S. Dewan, and S. Birchfield, "Fast-FoundationStereo: Real-Time Zero-Shot Stereo Matching," *arXiv preprint arXiv:2512.11130*, 2026. Available: https://arxiv.org/abs/2512.11130

[5] NVlabs, "FoundationStereo," GitHub Repository, 2025. [Online]. Available: https://github.com/NVlabs/FoundationStereo

[6] NVIDIA Corporation, "Isaac ROS DNN Stereo Depth," NVIDIA Developer Documentation (Release 4.0), 2024. [Online]. Available: https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_dnn_stereo_depth/index.html

[7] B. Wen, M. Trepte, O. Gallo, J. Aribido, S. Birchfield, and J. Kautz, "FoundationStereo: Zero-Shot Stereo Matching," *arXiv preprint arXiv:2501.09898v4 [cs.CV]*, Apr. 2025. Available: https://arxiv.org/abs/2501.09898

[8] NVIDIA Corporation, "Isaac ROS Mapping And Localization: Tutorial Map Creation," NVIDIA Developer Documentation (Release 4.0), 2024. [Online]. Available: https://nvidia-isaac-ros.github.io/v/release-4.0/repositories_and_packages/isaac_ros_mapping_and_localization/isaac_mapping_ros/tutorial_map_creation.html

[9] NVIDIA Corporation, "FoundationStereo," NVIDIA NGC Catalog, Deployable v2.0. [Online]. Available: https://catalog.ngc.nvidia.com/orgs/nvidia/teams/tao/models/foundationstereo?version=deployable_v2.0

[10] S. Izadi, D. Kim, O. Hilliges, D. Molyneaux, R. Newcombe, P. Kohli, J. Shotton, S. Hodges, D. Freeman, A. Davison, and A. Fitzgibbon, "KinectFusion: Real-time 3D Reconstruction and Interaction Using a Moving Depth Camera," *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pp. 559-568, 2011. Available: https://dl.acm.org/doi/10.1145/2047196.2047270

[11] C. Taglione, C. M. Mateo, and C. Stolz, "MMPNeRF: Multi-Modal Neural Rendering of Transparent Objects Using Color and Polarimetric Images," in *2024 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2024. doi:

10.1109/ROBIO64047.2024.10907535. Available: https://ieeexplore.ieee.org/document/10907535

[12] L. Yang et al., "Depth-Anything," GitHub Repository, 2024. [Online]. Available: https://github.com/DepthAnything/Depth-Anything-V2

[13] Kujiale-Vision, "MoGe: Monocular Geometry Foundation Model," GitHub Repository, 2024. [Online]. Available: https://github.com/microsoft/MoGe

[14] IDEA-Research, "Grounded-Segment-Anything," GitHub Repository, 2024. [Online]. Available: https://github.com/IDEA-Research/Grounded-Segment-Anything